Adam Lee
Prof. Levent Yilmaz
COMP3270
2/1/2026

Homework 1

1. Worst-Case Running Time of find2D:
Worst-case time is O(n^2) since each of n rows is searched in O(n). It can not be linear.

2. 8-Queens Problem Specification:
The Input is Empty 8x8 board.
Representation Array Q[1 to 8], Q[i] = (row, column).
Our Constraints are that we have no same row, column, diagonal.
Output: Valid queen positions.

3. Missing Number Strategy:
Compute expected sum n(n-1)/2 and subtract array sum.
4. String to Integer:
Multiply * by 10 and then add +1 digit.

5. Maximum Difference:
Find min and max. Difference = max - min.
Pairwise method uses n(n-1)/2 pairs.

6. Mystery Algorithm:
Finds the maximum sum of any contiguous subarray.
n = 10, example array:
A = [2, -4, 3, 5, -1, 2, -6, 4, 2, -1]
Index:1 2 3 4 5 6 7 8 9 10
This algorithm brute forces till it find the max in the subarray, computes the sum, and keeps the largest, then prints out the following:
A = [2, -4, 3, 5, -1, 2, -6, 4, 2, -1]

Time: O(n^3).

7. Boggle Strategy 2:
The second strategy checks each dictionary word against the board using DFS.
Its time complexity is $O(m \cdot n^2 \cdot 8^n)$.
The first strategy is more efficient because it avoids repeated searches.
Using a Trie and prefix checking greatly improves performance.
With these optimizations, Strategy 2 becomes as efficient as Strategy 1.

8. Approximate Complexity:

O(n^2).

Step	Big-Oh complexity
1	O(1)
2	O(n)
3	O(n)
4	O(1)
5	O(n)
6	O(1)
7	O(1)
8	O(1)
9	O(1)
Complexity of the Algorithm	O(n^3)

9. Detailed Complexity:

T(n)=an^2+bn+c.

Step	Cost of each execution	Total # of times executed
1	c_1	1
2	c_2	n
3	c_3	n(n+1)/2
4	c_4	n(n+1)/2
5	c_5	n(n+1)(n+2)/6
6	c_6	n(n+1)(n+2)/6
7	c_7	n(n+1)/2
8	c_8	n(n+1)/2
9	c_9	1

Answer: T(n)=an^3+bn^2+cn+d = O(n^3).

10. Consecutive Count:

Algorithm is incorrect; fails on edge cases. Its skips overlapping pairs,

Example: if input is: 1 1 1

Then Correct answer = 2, and algorithm outputs: 1.

11. Induction Proof:
f(n)=3^n-2^n, n < 0 proven by induction.

12.
Loop Invariant: max contains the largest value among the elements
A[1], A[2], ... , A[i−1].
Initialization: Before the loop starts: max = A[1]
Only the first element has been examined, so max is the largest in: A[1..1]
Maintenance: max is the largest in A[1..i−1]
During iteration i: If A[i] > max, then max is updated to A[i], Otherwise, max stays the same
In both cases, after the update: max is the largest in A[1..i]
Termination: The loop ends when: i = n + 1
By the loop invariant: max is the largest in A[1..n]

13. Recursive Reverse:
Pseudocode for part A:

```
Reverse(A, p, q):
if p >= q then
    return
swap(A[p], A[q])
        Reverse(A, p+1, q-1)
```

Pseudocode for part B:
Reverse(1,8): i<33270! -> !<33270i

Reverse(2,7): !<33270i -> !033270i

Reverse(3,6): !033270i -> !037230i

Reverse(4,5): !037230i -> !037320i

Reverse(5,4): Base case -> !037320i

Final Output: !037320i